UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843

Special Topic in Data Engineering

Tutorial 4: GenAI

LECTURER: DR. ARYATI BINTI BAKRI

NO.	NAME	MATRIC NUMBER
1.	IMAN ABADI BINN MOHD NIZWAN	A23CS0084
2.	MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

1. AI-Assisted Data Engineering

AI-Assisted Data Engineering refers to the integration of artificial intelligence and machine learning techniques into the design, development and management of data pipelines and infrastructure. It is a specialized method of building and managing pipelines, infrastructure and systems designed to support AI and machine learning applications which is different from conventional data engineering where it focuses on moving structured data for reporting and business intelligence. AI-assisted data engineering extends this scope to handle multi-modal unstructured data such as text and images, real-time streaming, model training pipelines, vector databases and Large Language Model (LLM) workflows (Medium, 2025).

An example of AI-assisted data engineering is machine learning (ML) pipeline development. AI data engineering orchestrates the full machine learning lifecycle from feature engineering and training data preparation through to model deployment, real-time inference and continuous monitoring. Another example is unstructured data processing for text embedding generation, image preprocessing and high-throughput IoT sensor ingestion all demand specialized tooling Hugging Face Transformers for NLP or Apache Kafka for event streaming (Medium, 2025).

2. AI Agent used in the Tutorial

The AI used in this tutorial is Claude, made by Anthropic. The group used it through Claude Code, Anthropic's agentic coding tool, running inside the Claude Desktop application. Claude Code is more than a chatbot, it can read and write files in the project folder, run terminal commands, read the output and fix its own errors. It runs on one of Anthropic's Claude models, which is Claude Opus 4.7 in this tutorial.

2.1 Claude's Role

Claude acted as a coding collaborator for the project. The group described each task in plain English, and Claude turned those instructions into working code, ran it, and helped correct any errors. In this way it handled the routine coding work, while the group stayed in charge of the design decisions and the checking of results. Anthropic also offers a Data Engineering plugin for Claude Code, which adds data-focused skills such as warehouse exploration, pipeline authoring, and workflow management to the coding environment.

2.2 How Claude Operated

Each stage followed the same simple loop. The group wrote a prompt describing the stage, including its inputs, outputs, and rules. Claude then made a short plan and asked a question if anything was unclear. Once the plan was approved, Claude wrote the code and ran it. The group reviewed the result and requested fixes where needed.

3. Case Study

This section presents a practical case study that applies the data pipeline concepts discussed earlier to a real-world scenario. By tracking the ingestion, transformation, and storage of public environmental data, this case study serves as a concrete blueprint for building automated, production-grade data workflows.

3.1 Background

This project implements that pipeline pattern end-to-end on a real public dataset using the Medallion architecture (Bronze / Silver /Gold). The dataset used is the NYC Air Quality Surveillance CSV published by the New York City Department of Health. It contains around 19,000 rows covering 18 pollution-related indicators (PM2.5, NO_2, Ozone, asthma rates, boiler emissions, vehicle miles travelled) measured across NYC neighbourhoods from 2008 to 2023.

3.2 Objective

- Quantify the long-term PM2.5 trend at the city level (2008–2023).
- Rank neighborhoods by exposure, so remediation effort can be targeted.
- Classify every measurement against EPA health bands (Good / Moderate / Unhealthy / Hazardous).
- Make the analysis reproducible and re-runnable, because the dataset is updated over time and the answer will need to be refreshed.

3.3 Use Case

The use case diagram in *Figure 3.1* is the NYC Air Quality Pipeline System contains three actors and six use cases. The three actors are the Data Engineer, Claude AI Agent and the NYC Open Data Portal. The six use cases follow the Medallion architecture which contains ingest raw dataset (bronze), profile & clean data (silver), transform & classify (gold), validate and load, visualise results and orchestrate pipeline. The data engineer and Claude are connected to all pipeline-stage use cases where they act as the initiator who writes each prompt and reviews each output that Claude writes and implements.. The NYC Open Data Portal connects solely to ingest raw dataset, as it is the external source that supplies the 2.2 MB CSV file consumed in Phase 4.1.

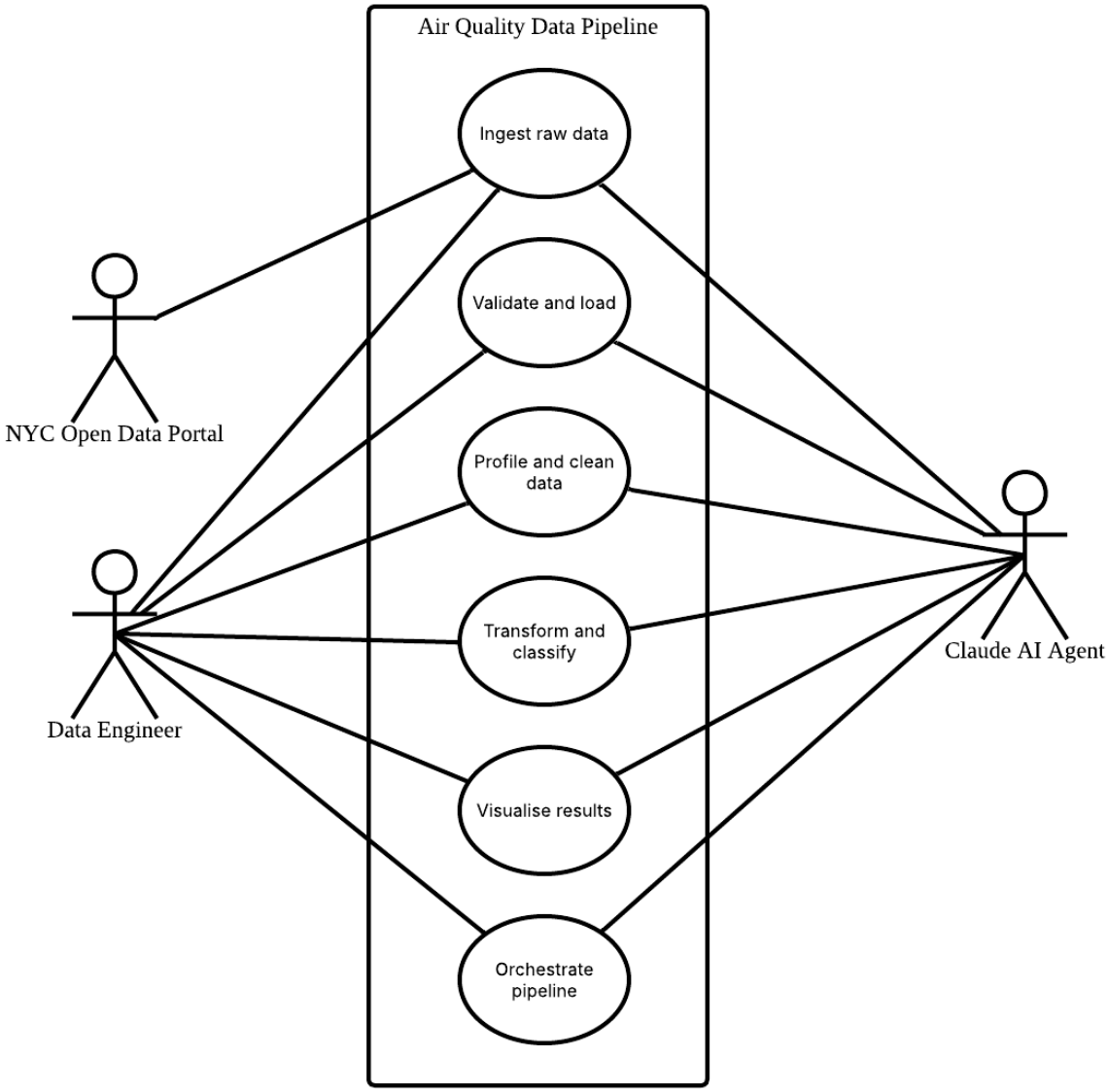


Figure 3.1 Use Case Diagram of Air Quality Data Pipeline

4. Pipeline Implementation: Prompt and Code

This section walks through every prompt given to Claude during the construction of this pipeline in order. For each prompt, the text the user typed and Claude's responses are recorded with notes on the adaptations or issues encountered as well as the result produced. Six prompts in total produced the entire working pipeline.

4.1 Data Ingestion Stage

Figure 4.1 shows the first prompt used asking Claude to build the directory tree from scratch and write the ingest stage. Plan mode was active, so Claude produced a written plan before touching any files before the actual implementation was executed.

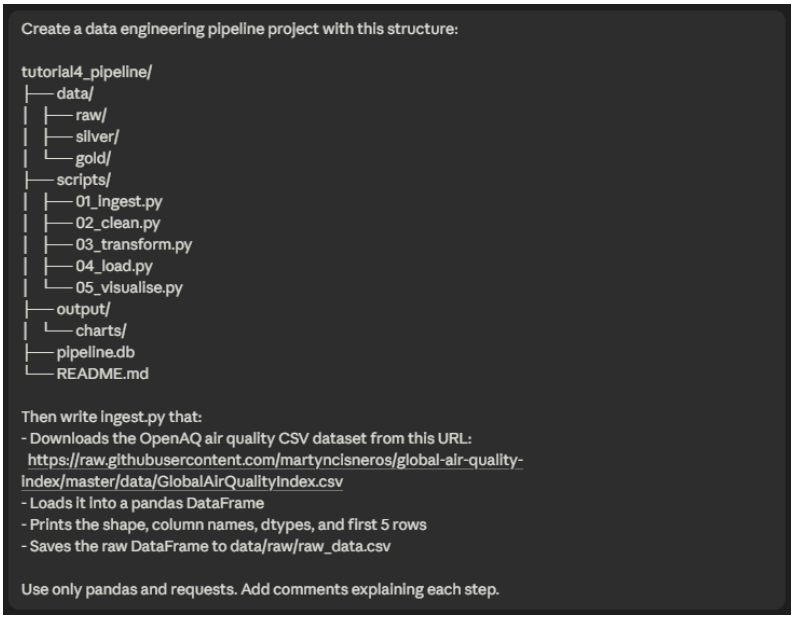


Figure 4.1 Ingestion Stage Prompt

Claude initiated the project by mapping out a structured plan, building the exact directory tree with .gitkeep files, and writing a comprehensive setup README.md. It fully implemented 01_ingest.py which includes robust HTTP error handling, memory-efficient CSV processing and the required data inspections shown in *Figure 4.2*.

```python
# Source CSV: NYC Air Quality Surveillance Data. Small enough to fetch into memory.
DATASET_URL = "https://data.cityofnewyork.us/api/views/c3uy-2p5r/rows.csv"

# Resolve paths relative to the script so the pipeline runs from any CWD.
PROJECT_ROOT = Path(__file__).resolve().parent.parent
RAW_OUTPUT_PATH = PROJECT_ROOT / "data" / "raw" / "raw_data.csv"


def main() -> None:
    # 1. Fetch the CSV over HTTP. timeout guards against a hung connection.
    print(f"Downloading dataset from {DATASET_URL} ...")
    response = requests.get(DATASET_URL, timeout=30)
    # Surface HTTP errors (404, 500, ...) instead of silently parsing an error page.
    response.raise_for_status()

    # 2. Parse the CSV from the in-memory response body via StringIO,
    #    so we don't need an intermediate temp file before pandas sees it.
    df = pd.read_csv(StringIO(response.text))

    # 3. Inspect the DataFrame.
    print("\n--- Shape ---")
    print(df.shape)

    print("\n--- Columns ---")
    print(df.columns.tolist())

    print("\n--- Dtypes ---")
    print(df.dtypes)

    print("\n--- Head (first 5 rows) ---")
    print(df.head(5))

    # 4. Make sure data/raw/ exists, then persist.
    #    index=False so re-reading later does not produce a phantom "Unnamed: 0" column.
    RAW_OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(RAW_OUTPUT_PATH, index=False)

    print(f"\nSaved raw dataset to: {RAW_OUTPUT_PATH}")
```

Figure 4.2 Ingestion Stage Code

The initial dataset URL returned a 404 error and Claude handled the situation by pausing the implementation to offer alternative data sources instead shown in *Figure 4.3*. Once the user selected the NYC Air Quality dataset, the ingestion script was successfully updated and verified, downloading a 2.2 MB raw data file to the local directory which was run automatically by Claude.

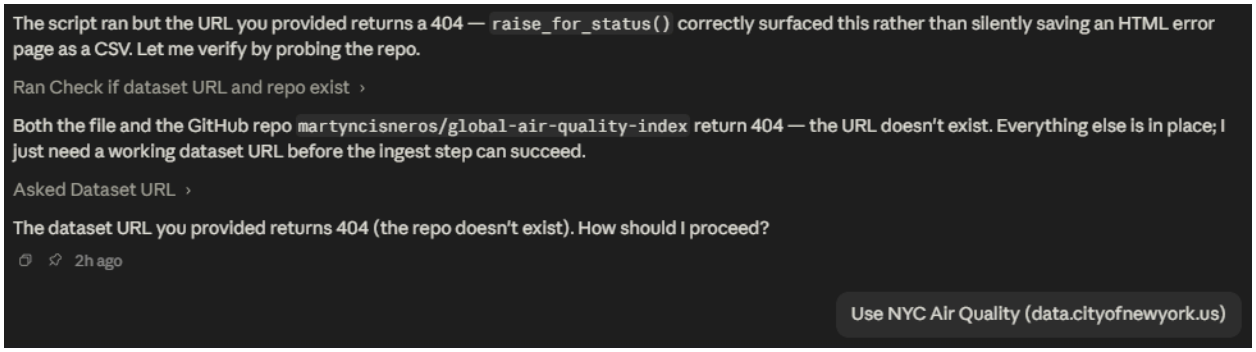


Figure 4.3 Claude's 404 Error Handling Response

4.2 Cleaning Stage

After the ingestion stage, the next prompt asked for the cleaning stage. The prompt was unusually detailed. It specified the profiling step, every cleaning rule and the before and after summary format.

Figure 4.4 shows a prompt that was used in Claude to clean the data. Claude fixed the data cleaning steps by putting them in a strict, smart order.

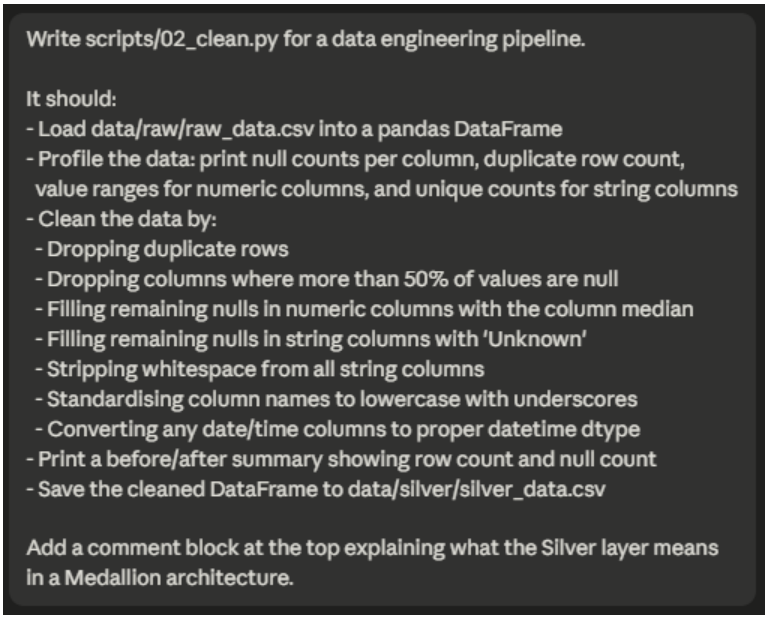
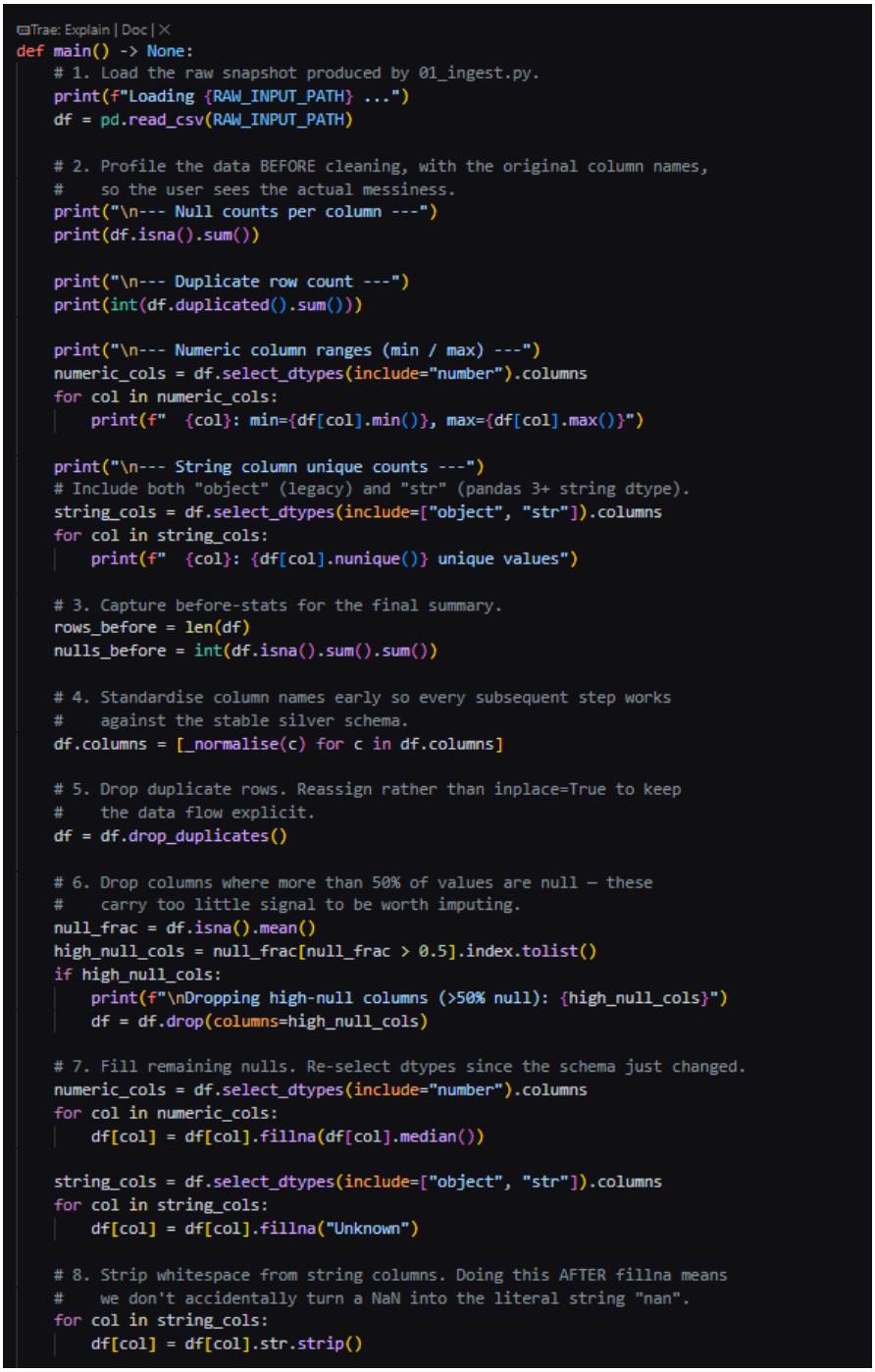


Figure 4.4 Cleaning Stage Prompt

Figure 4.5 shows code implementation for the cleaning stage. First, Claude looked at the raw data and saved the early stats. Then, Claude cleaned up the column names by turning spaces and symbols into underscores and lowercasing everything like changing *Unique ID* to *unique_id*. After that, Claude removed duplicate rows, deleted columns with too many empty spaces, filled in any remaining blank spots, cleaned up extra whitespace and converted date columns into actual date formats. To make sure Claude did not accidentally change the wrong columns, it only converted columns with "date" in their name if most of the rows matched a real date. In the end, Claude's script dropped one completely empty column called "Message," changed start_date to a proper date format and saved a clean new file with 18,862 rows and absolutely no missing data.



```
Trae: Explain | Doc | ×
def main() -> None:
    # 1. Load the raw snapshot produced by 01_ingest.py.
    print(f"Loading {RAW_INPUT_PATH} ...")
    df = pd.read_csv(RAW_INPUT_PATH)

    # 2. Profile the data BEFORE cleaning, with the original column names,
    #    so the user sees the actual messiness.
    print("\n--- Null counts per column ---")
    print(df.isna().sum())

    print("\n--- Duplicate row count ---")
    print(int(df.duplicated().sum()))

    print("\n--- Numeric column ranges (min / max) ---")
    numeric_cols = df.select_dtypes(include="number").columns
    for col in numeric_cols:
        print(f"  {col}: min={df[col].min()}, max={df[col].max()}")

    print("\n--- String column unique counts ---")
    # Include both "object" (legacy) and "str" (pandas 3+ string dtype).
    string_cols = df.select_dtypes(include=["object", "str"]).columns
    for col in string_cols:
        print(f"  {col}: {df[col].nunique()} unique values")

    # 3. Capture before-stats for the final summary.
    rows_before = len(df)
    nulls_before = int(df.isna().sum().sum())

    # 4. Standardise column names early so every subsequent step works
    #    against the stable silver schema.
    df.columns = [_normalise(c) for c in df.columns]

    # 5. Drop duplicate rows. Reassign rather than inplace=True to keep
    #    the data flow explicit.
    df = df.drop_duplicates()

    # 6. Drop columns where more than 50% of values are null — these
    #    carry too little signal to be worth imputing.
    null_frac = df.isna().mean()
    high_null_cols = null_frac[null_frac > 0.5].index.tolist()
    if high_null_cols:
        print(f"\nDropping high-null columns (>50% null): {high_null_cols}")
        df = df.drop(columns=high_null_cols)

    # 7. Fill remaining nulls. Re-select dtypes since the schema just changed.
    numeric_cols = df.select_dtypes(include="number").columns
    for col in numeric_cols:
        df[col] = df[col].fillna(df[col].median())

    string_cols = df.select_dtypes(include=["object", "str"]).columns
    for col in string_cols:
        df[col] = df[col].fillna("Unknown")

    # 8. Strip whitespace from string columns. Doing this AFTER fillna means
    #    we don't accidentally turn a NaN into the literal string "nan".
    for col in string_cols:
        df[col] = df[col].str.strip()
```

Figure 4.5 Cleaning Stage Code

4.3 Transformation Stage

The transform stage was where the dataset mismatch became most visible. *Figure 4.6* shows the prompt used assuming a multi-country Air Quality IndexAQI dataset but the actual data is NYC

only with no country, no real city and 18 different pollution indicators in incompatible units. This is due to the change of the dataset that was not taken into account earlier.

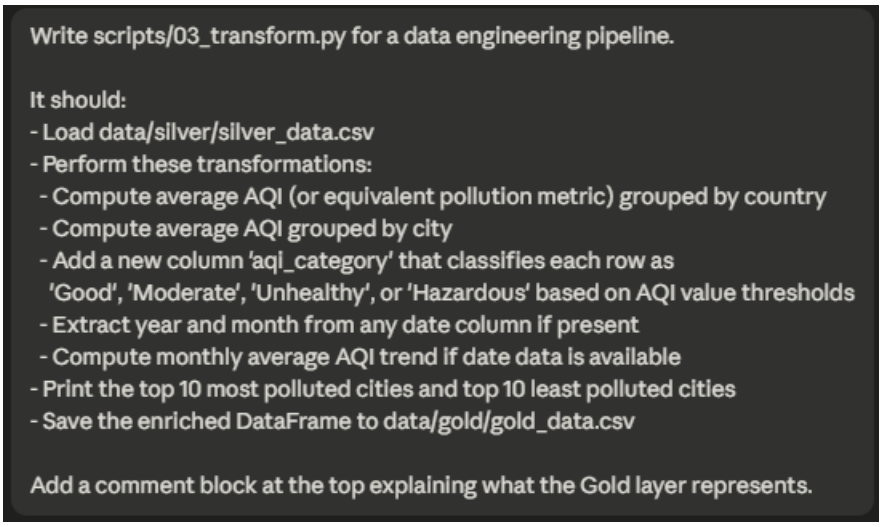


Figure 4.6 Transformation Stage Prompt

Claude designed three documented semantic adaptations to utilise the NYC specific dataset with the project requirements where it maps country to geo_type_name, city to geo_place_name, and filtering specifically for PM2.5 to serve as the AQI metric. *Figure 4.7* shows Claude implemented vectorized, EPA-aligned AQI thresholds using pd.cut, extracted year and month dimensions, and computed localized pollution trends. Claude also resolved a

The process generated gold_data.csv, which successfully passed verification. The final dataset confirmed a clear downward trend in pollution from 2008 to 2023, with all NYC records falling strictly into the "Good" and "Moderate" categories. Geographically, the data validated expectations, identifying Manhattan core neighborhoods like Midtown and Chelsea as the most polluted, while outer waterfront areas like the Rockaways remained the cleanest.

```python
def main() -> None:
    # 1. Load Silver. parse_dates restores the datetime dtype lost on CSV round-trip.
    print(f"Loading {SILVER_INPUT_PATH} ...")
    df = pd.read_csv(SILVER_INPUT_PATH, parse_dates=["start_date"])

    # 2. Filter to the AQI proxy. Bail loudly if it's missing — protects
    #    against future data drift where the indicator name might change.
    if AQI_INDICATOR not in df["name"].unique():
        raise ValueError(
            f"Expected indicator '{AQI_INDICATOR}' not found in silver data. "
            f"Available indicators: {sorted(df['name'].unique())}"
        )
    df = df[df["name"] == AQI_INDICATOR].copy()
    print(f"\nFiltered to indicator: '{AQI_INDICATOR}' ({len(df)} rows)")

    # 3. Aggregations. Printed only — they're summaries, not row-level data.
    #    "by region" is the country substitution; country isn't meaningful
    #    for an NYC-only dataset.
    print("\n--- Average AQI by region (geo_type_name) ---")
    print("  Note: 'country' is not applicable - all rows are NYC.")
    print(df.groupby("geo_type_name")["data_value"].mean().round(2))

    print("\n--- Average AQI by city (geo_place_name) - first 15 ---")
    avg_by_city = df.groupby("geo_place_name")["data_value"].mean().round(2)
    print(avg_by_city.head(15))

    # 4. Add aqi_category via pd.cut. Vectorised, preserves order, and
    #    the labels become a Categorical so downstream filtering is cheap.
    df["aqi_category"] = pd.cut(
        df["data_value"], bins=AQI_BINS, labels=AQI_LABELS, right=False
    )

    # 5. Extract year and month — Silver guarantees start_date is datetime64.
    df["year"] = df["start_date"].dt.year
    df["month"] = df["start_date"].dt.month

    # 6. Monthly average trend.
    print("\n--- Monthly average AQI trend (year, month) ---")
    monthly_trend = (
        df.groupby(["year", "month"])["data_value"].mean().round(2).sort_index()
    )
    print(monthly_trend)

    # 7. Top-10 most polluted and top-10 least polluted cities, by mean PM2.5.
    sorted_by_city = avg_by_city.sort_values(ascending=False)
    print("\n--- Top 10 most polluted cities ---")
    print(sorted_by_city.head(10))
    print("\n--- Top 10 least polluted cities ---")
    print(sorted_by_city.tail(10).sort_values())

    # 8. Persist the row-level enriched DataFrame. The aggregations above
    #    are summaries; Stage 4 can re-aggregate from this row-level table.
    GOLD_OUTPUT_PATH.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(GOLD_OUTPUT_PATH, index=False)
    print(f"\nSaved enriched dataset to: {GOLD_OUTPUT_PATH}")
```

Figure 4.7 Transformation Stage Code

4.4 Loading Stage

The load stage is when data moves from normal files on a computer into a database which is the data warehouse of this system where people can search and query it. To make sure the data is safe and correct, the pipeline runs four checks before the data is loaded and one final check after it is loaded.

Figure 4.8 shows a prompt that was used in Claude to load the data into the data warehouse. Claude set up the final step to safely move the cleaned data into a searchable SQLite database table called *gold_air_quality*.

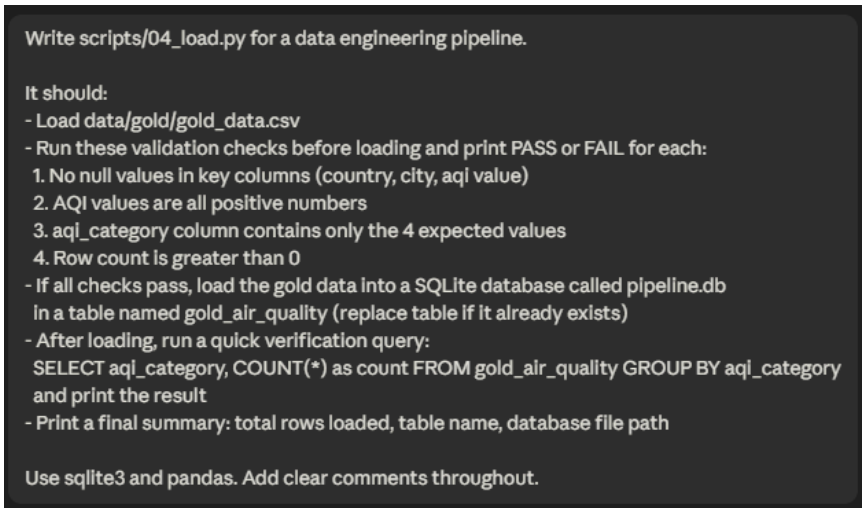


Figure 4.8 Loading Stage Prompt

Figure 4.9 shows the code implementation of validation before entering the database. First, Claude matched the general column names requested in the instructions to the actual New York City data columns, like mapping the air quality value to *data_value*. To protect the database from bad data, Claude created a unified system that runs four safety checks which are it ensures there are no missing values in key columns, checks that all air quality numbers are positive, confirms the safety categories match allowed words like "Good" or "Moderate," and verifies the file is not empty. If any check fails, the script immediately stops to keep the database safe.

```python
Trae: Explain | Doc | ×
def validate(df: pd.DataFrame) -> bool:
    print("Running validation checks ...")
    results = []

    # Check 1: no nulls in key columns.
    null_counts = df[KEY_COLUMNS].isna().sum()
    results.append(
        _report(
            "No nulls in key columns (geo_type_name, geo_place_name, data_value)",
            bool((null_counts == 0).all()),
            detail=f"nulls={null_counts.to_dict()}" if (null_counts != 0).any() else "",
        )
    )

    # Check 2: AQI (data_value) values are all positive numbers.
    aqi = df["data_value"]
    all_positive = pd.api.types.is_numeric_dtype(aqi) and bool((aqi > 0).all())
    results.append(
        _report(
            "AQI values are all positive numbers",
            all_positive,
            detail=f"min={aqi.min()}" if not all_positive else "",
        )
    )

    # Check 3: aqi_category contains only the 4 expected labels.
    observed = set(df["aqi_category"].dropna().unique())
    unexpected = observed - EXPECTED_CATEGORIES
    results.append(
        _report(
            "aqi_category contains only the 4 expected values",
            not unexpected,
            detail=f"unexpected={unexpected}" if unexpected else "",
        )
    )

    # Check 4: row count is greater than 0.
    results.append(
        _report(
            "Row count is greater than 0",
            len(df) > 0,
            detail=f"rows={len(df)}",
        )
    )

    return all(results)
```

Figure 4.9 Lading Stage Validation Code

Figure 4.10 shows the code implementation of the loading stage. Since everything passed, the code overwrites the old database table with new data every time the pipeline runs, saving the dates in a proper time format.

```python
def main() -> None:
    # 1. Load the Gold CSV. parse_dates keeps start_date as datetime so
    #    it lands in SQLite as a proper ISO string rather than the
    #    pre-stringified form.
    print(f"Loading {GOLD_INPUT_PATH} ...")
    df = pd.read_csv(GOLD_INPUT_PATH, parse_dates=["start_date"])

    # 2. Validate. If any check fails, refuse to load — we'd rather
    #    leave pipeline.db untouched than ingest bad data.
    if not validate(df):
        print("\nValidation failed - aborting load. pipeline.db left untouched.")
        sys.exit(1)
    print("\nAll checks passed.")

    # 3. Load into SQLite. if_exists='replace' gives us idempotent reruns:
    #    the table is dropped and recreated each time.
    #    sqlite3.connect() creates pipeline.db if it doesn't already exist.
    print(f"\nWriting {len(df)} rows to '{TABLE_NAME}' in {DB_PATH} ...")
    with sqlite3.connect(DB_PATH) as conn:
        df.to_sql(TABLE_NAME, conn, if_exists="replace", index=False)

        # 4. Verification query - read back what we just wrote, grouped
        #    by category, to confirm the round-trip looks right.
        print("\n--- Verification: row count by aqi_category ---")
        verify_df = pd.read_sql_query(
            f"SELECT aqi_category, COUNT(*) AS count "
            f"FROM {TABLE_NAME} GROUP BY aqi_category",
            conn,
        )
        print(verify_df.to_string(index=False))

    # 5. Final summary.
    print("\n--- Load summary ---")
    print(f"  Total rows loaded: {len(df)}")
    print(f"  Table name:        {TABLE_NAME}")
    print(f"  Database file:     {DB_PATH}")
```

Figure 4.10 Lading Stage Code

4.5 Serving Stage

Figure 4.11 shows the prompt for visualization. Claude set up the data visualization step by first configuring Matplotlib to run in the background without opening a popup window, ensuring the script can run smoothly on Windows. To make all three charts look uniform, Claude applied a consistent Seaborn design theme and created a shared helper function that neatly formats the titles, saves the images clearly and frees up computer memory afterward.

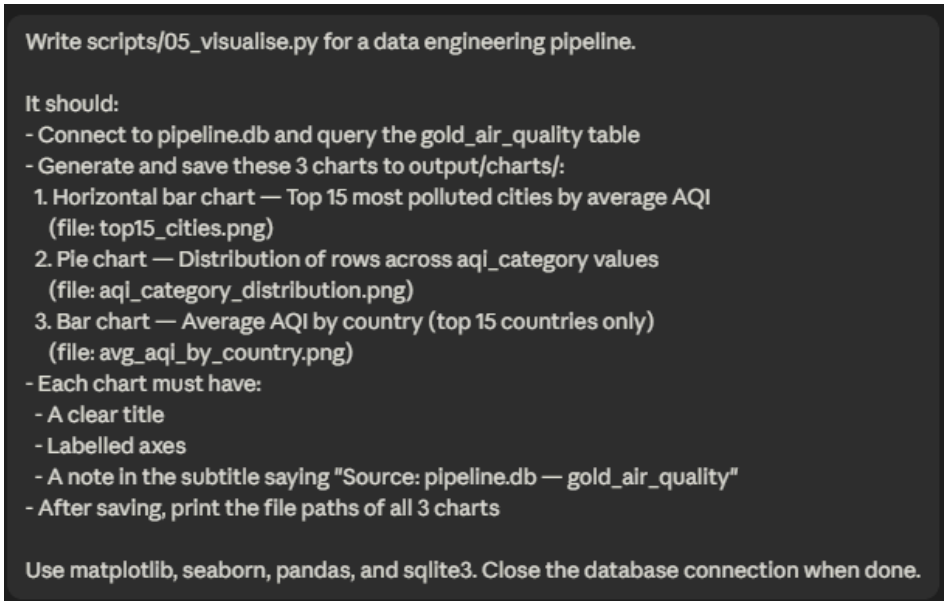


Figure 4.11 Serving Stage Prompt

Figure 4.12 shows the code implementation for visualization. First, Claude made a horizontal bar chart showing the 15 most polluted neighborhoods, fixing an initial sorting issue so the worst area correctly appears at the very top. Next, Claude created a pie chart showing the breakdown of air quality categories, complete with clear borders and labels for both percentages and counts. The third chart was designed to show average air quality by region, including a special note explaining that country-level data is not available since this is NYC-only data. After resolving an initial error by installing the missing Matplotlib and Seaborn tools, Claude safely connected to the SQLite database and successfully generated all three final images.

```python
def main() -> None:
    # 1. Pull the gold table out of SQLite. The `with` block closes the
    #    connection automatically once we leave it.
    print(f"Connecting to {DB_PATH} ...")
    with sqlite3.connect(DB_PATH) as conn:
        df = pd.read_sql_query(f"SELECT * FROM {TABLE_NAME}", conn)
    print(f"Loaded {len(df)} rows from '{TABLE_NAME}'.")

    # 2. Make sure the output dir exists.
    CHARTS_DIR.mkdir(parents=True, exist_ok=True)

    # 3. Generate the three charts.
    paths = {
        "top15_cities": CHARTS_DIR / "top15_cities.png",
        "aqi_category_distribution": CHARTS_DIR / "aqi_category_distribution.png",
        "avg_aqi_by_country": CHARTS_DIR / "avg_aqi_by_country.png",
    }
    chart_top15_cities(df, paths["top15_cities"])
    chart_aqi_category_distribution(df, paths["aqi_category_distribution"])
    chart_avg_aqi_by_country(df, paths["avg_aqi_by_country"])

    # 4. Report what we wrote.
    print("\nSaved charts:")
    for key, p in paths.items():
        print(f"  {key}: {p}")
```

Figure 4.12 Serving Stage Code Implementation

4.6 Pipeline Orchestration

The final coding prompt is for a single entry point that runs all five stages in order shown in *Figure 4.13*

> Write a run_pipeline.py script in the root of the project that runs all 5 pipeline scripts in order (01 through 05) using subprocess.
> For each script, print a header like "=== Running Phase 1: Ingest ===" before running it, and print "=== Phase 1 Complete ===" after.
> If any script fails, stop the pipeline and print the error message clearly.

Figure 4.12 Pipeline Orchestration Prompt

Figure 4.13 shows the developed run_pipeline.py at the project root to serve as an independent master orchestrator, defining an ordered list to execute all five pipeline stages sequentially. By utilizing subprocess.run with sys.executable, the orchestrator ensures virtual-environment consistency and isolates each script's execution state while streaming progress live to the

terminal. End-to-end verification confirmed that all five phases run successfully in sequence, with terminal output filtering validating that the phase boundaries and the final pipeline success confirmation in specified order.

```python
PHASES = [
    (1, "Ingest",     "01_ingest.py"),
    (2, "Clean",      "02_clean.py"),
    (3, "Transform",  "03_transform.py"),
    (4, "Load",       "04_load.py"),
    (5, "Visualise",  "05_visualise.py"),
]


def main() -> None:
    for number, name, filename in PHASES:
        script_path = SCRIPTS_DIR / filename

        print(f"\n=== Running Phase {number}: {name} ===\n")

        # Use sys.executable so the child uses the same Python interpreter
        # as the orchestrator. check=False because we want to handle the
        # error ourselves with a clear message instead of a raw traceback.
        result = subprocess.run([sys.executable, str(script_path)], check=False)

        if result.returncode != 0:
            print(
                f"\n!!! Pipeline aborted at Phase {number}: {name} !!!\n"
                f"    Script:      {script_path}\n"
                f"    Exit code:   {result.returncode}\n"
                f"    See the output above for the underlying error.",
                file=sys.stderr,
            )
            sys.exit(result.returncode)

        print(f"\n=== Phase {number} Complete ===")

    print("\nAll 5 phases completed successfully.")
```

Figure 4.13 Pipeline Orchestration Code Implementation

5. Results and Discussion

Top 15 Neighbourhoods by Average PM2.5

Figure 5.1 shows the top 15 neighbourhoods ranked by their average PM2.5 level. Midtown recorded the highest average at around 12.8 mcg/m³, followed by other dense Manhattan areas such as Gramercy and Chelsea. This suggests that areas with heavier traffic and commercial activity tend to have worse air quality compared to the rest of the city.

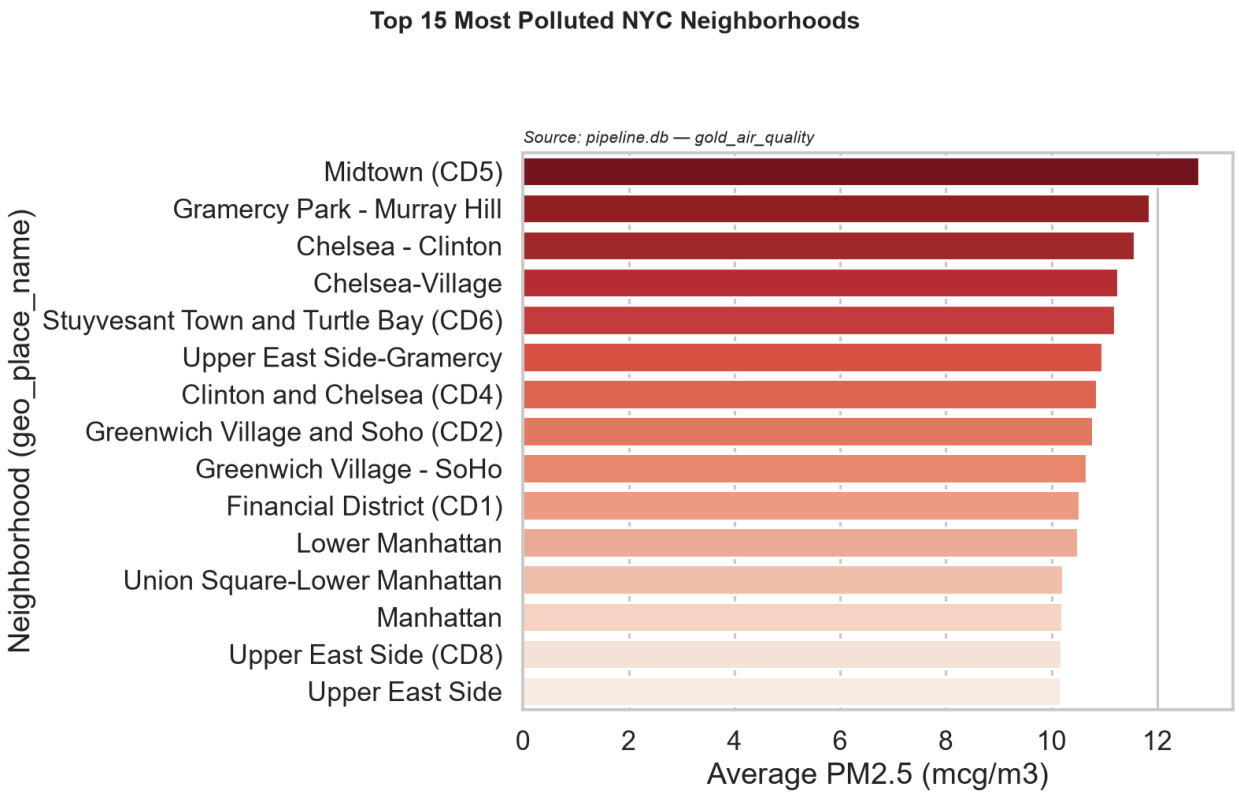


Figure 5.1 Top 15 Neighbourhoods by Average PM2.5

Average PM2.5 by Region Type

Figure 5.2 shows the distribution of AQI categories across all recorded readings. Around 90.6% of readings fall into the Good category, while the remaining 9.4% fall into Moderate. This indicates that New York City's air quality is generally acceptable over the period studied, with only a small portion of readings reaching a level that may cause concern for sensitive groups.

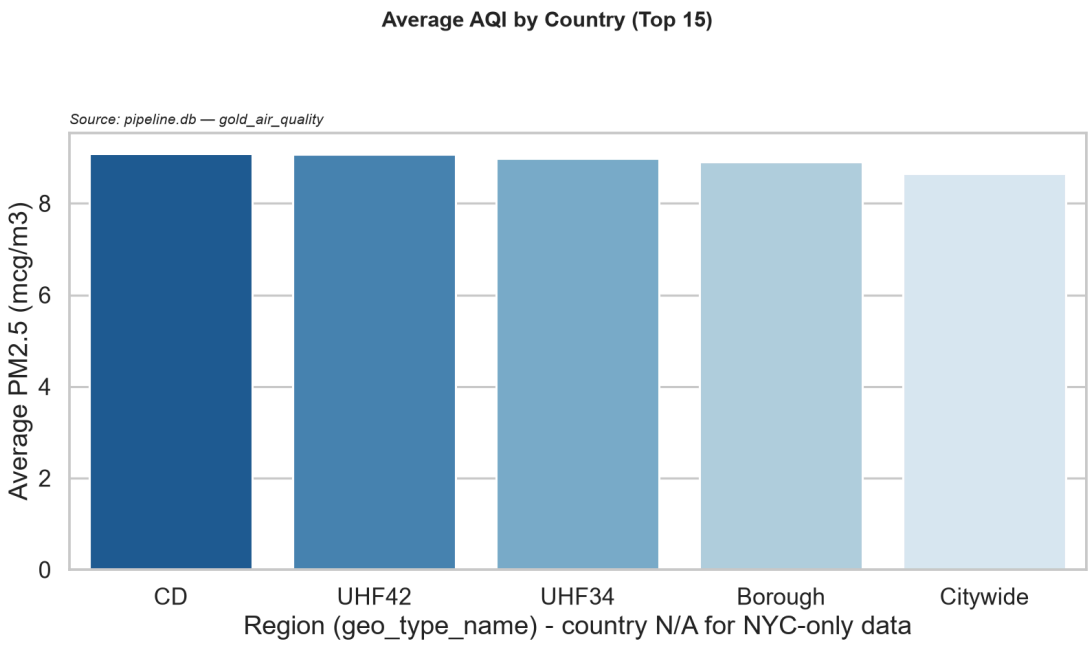


Figure 5.2 Average PM2.5 by Region Type

AQI Category Distribution

Figure 5.3 compares the average PM2.5 level across different region types. Community districts recorded the highest average at around 9.09 mcg/m³, while the citywide average sits at around 8.65 mcg/m³. This shows that smaller, localised areas tend to have slightly higher pollution levels than the city as a whole, which is expected given the variation between dense urban neighbourhoods and open outer areas.

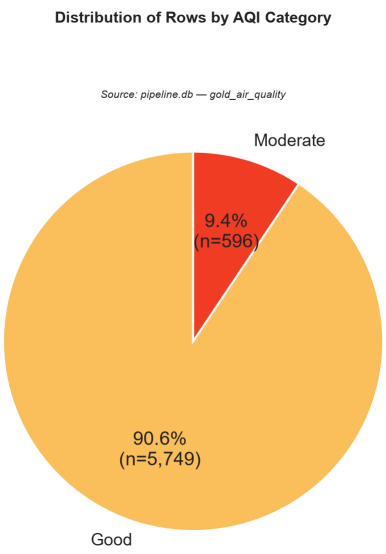


Figure 5.3 AQI Category Distribution

6. Reflection

Iman Abadi bin Mohd Nizwan

In this project, the basics of data engineering with AI were explored from start to finish. It was realized that the quality of the AI's work was directly driven by how well the instructions were given. Because of this, writing clear prompts was seen as a main, necessary skill rather than just a minor task.The advantages of using AI was most clearly seen when a broken website link was seen while gathering data. Instead of the project being stopped by the error, the problem was diagnosed and new data choices were offered by the AI on its own. This shows the potential AI has in debugging errors so that a pipeline can be fully automated by AI.

Mohamed Alif Fathi bin Abdul Latif

Through this tutorial, a better understanding of how a data pipeline works from start to finish was gained. It was observed how raw data is moved through each stage, from ingestion all the way to charts, and how each step is dependent on the one before it. The process was made faster with the help of Claude, but the importance of writing clear prompts was also realised. When the instructions given were vague, the output produced was not what had been intended. Going forward, more practice in writing better prompts is needed, along with a deeper understanding of how data can be validated properly at each stage of the pipeline.

7. References

Medium. (2025, September 2). *What is AI Data Engineering? The Complete Guide to the Future of Data Infrastructure*. https://medium.com/@averageguymedianow/what-is-ai-data-engineering-the-complete-guide-to-the-future-of-data-infrastructure-35eab6147aab